



Learning to Rank

IFN647 – Week 7

Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

Learning Objectives

Topic ID	Key topics	Mapping to slides
1	Learning to Rank	2-5
2	Query expansion	6-13
3	Semi-supervised learning for Learning to Rank	14-16
4	Rocchio-Based Information Filtering Model	17-21
5	Probabilistic Information Filtering Models	22-25
6	BM25-based IF model	26-27
7	LLM-based Information Filtering	28-30
	References	31

1. Learning to Rank

- **Learning to rank (machine-learned ranking)** is the application of machine learning, in the construction of [ranking models](#) for information retrieval, information filtering or recommender systems.
- A ranking function is typically defined as follows:
 - $f(u, d) \rightarrow \text{relevance_score}$
 - where:
 - u : a user
 - d : a document/item
- 🖱️ The objective is to rank incoming items or documents according to their personal relevance to a user (i.e., the user's information needs), where the ranking function assigns a relevance score to each document.
- The ranking function may be designed as an ad hoc model applicable to any user, or as an adaptive model that is tailored to a specific user over a given period.

Query-Based Ranking to User-Centric IF

- Traditional learning to rank for IR
 - $f(q, d) \rightarrow \text{ranking_score}$, where f is an IR model
- Extending learning to rank for IF
 - $f(u, d) \rightarrow \text{relevance_score}$
- Key Challenge: How to effectively capture and represent **user information needs**?

Approach	Query Expansion	Dual Encoders	Semi-Supervised Learning	Relevance model
Step 1	$q \rightarrow \hat{q}$	Encoder both q (or $\hat{q}$) and d	Obtain a small training set	Capture relevance feedback based q
Step 2	$f(\hat{q}, d) \rightarrow \text{ranking_score}$	$\cos(q, d) \rightarrow \text{Similarity_score}$	Learn a ranking model to generate additional pseudo-labels and enhance the training data.	Estimate the probability of the relevance of d to q

Dual-Encoder Framework for IF

- Instead of performing computationally expensive joint query–document interactions, the dual-encoder framework encodes queries and documents independently (e.g., using BERT-based embeddings) into dense vector representations, and then measures their similarity in a shared embedding space.
 - **Query encoder** - maps a query $q \rightarrow \vec{q}$
 - **Document encoder** - maps a document $d \rightarrow \vec{d}$
- Relevance is computed via a similarity function, such as cosine similarity or dot product.
- Dual-encoders are highly efficient, as document representations can be precomputed offline, enabling fast retrieval through approximate nearest neighbor search.
- The framework is also flexible: the “query” representation can be extended to encode richer user information needs, such as user profiles, context, or query expansions.
- However, due to the lack of fine-grained interactions between query and document tokens, dual-encoders may fail to capture subtle relevance signals.

Prompting Strategies for Ranking

- **Example prompt:**

Query: "UK: Britain's Channel 5 to broadcast Fashion Awards."

Rank the following documents from most to least relevant:

1. Royal Bank has three days to respond to the competing bid for London Insurance.
2. Canal Plus has concluded an accord in principle for it to buy Time-Warner's Castle Rock film studio.
3. GOLF-BRITISH OPEN FOURTH ROUND LEADERBOARD

🗨️ Ask ChatGPT

Ranked from most to least relevant:

1. Document 2 – Canal Plus media acquisition (closest to broadcasting context)
2. Document 1 – Royal Bank insurance news
3. Document 3 – Golf leaderboard

2. Query Expansion

- A variety of *automatic* or *semi-automatic* query expansion techniques have been developed
 - The goal is to improve effectiveness by matching related terms
 - Semi-automatic techniques require user interaction to select best expansion terms
 - The key to effective expansion is to choose words that are appropriate for the *context*, or topic, of the query.
 - For example, “aquarium” may be a good expansion term for “tank” in the query “tropical fish tanks”, but not appropriate for the query “armor for tanks”.
- Query suggestion is a related technique
 - alternative queries, not necessarily more terms

Query Expansion cont.

- Approaches usually based on an analysis of term co-occurrence
 - either in the entire document collection, a large collection of queries, or the top-ranked documents in a result list
 - query-based stemming also an expansion technique
- Automatic expansion based on general thesaurus (e.g., WordNet <https://wordnet.princeton.edu/>) not effective
 - does not take context into account (see the previous example for “aquarium”)
- To address this problem, people use **all** the words in a query to find related words rather than expanding each word separately.

Term Association Measures

- *Dice's Coefficient* for two words (or terms) a and b

$$\frac{2 \cdot n_{ab}}{n_a + n_b} \stackrel{\text{rank}}{=} \frac{n_{ab}}{n_a + n_b}$$

means that the formula is rank equivalent

- *Mutual Information*

$$\log \frac{P(a,b)}{P(a)P(b)} = \log N \cdot \frac{n_{ab}}{n_a \cdot n_b} \stackrel{\text{rank}}{=} \frac{n_{ab}}{n_a \cdot n_b}$$

- $P(a, b)$ is the probability that a and b occur in the same text window, and n_{ab} is the number of windows containing both words a and b .
- $P(a)$ is the probability that word a occurs in a text window, and n_a is the number of occurrences, $P(a) = n_a/N$, where a is a word, and
- N is the number of text windows in the collection.

Term Association Measures cont.

- Mutual Information measure favors low frequency terms
- *Expected Mutual Information Measure* (EMIM) addresses this problem by weighting the mutual information value using the probability $P(a, b)$.
 - It interested in the case where both terms occur, giving the formula:

$$P(a, b) \cdot \log \frac{P(a, b)}{P(a)P(b)} = \frac{n_{ab}}{N} \log \left(N \cdot \frac{n_{ab}}{n_a \cdot n_b} \right) \stackrel{rank}{=} n_{ab} \cdot \log \left(N \cdot \frac{n_{ab}}{n_a \cdot n_b} \right)$$

Term Association (χ^2) Measure

- *Pearson's Chi-squared (χ^2) measure*
 - compares the number of co-occurrences of two words with the expected number of co-occurrences if the two words were independent
 - normalizes this comparison by the expected number.
 - also limited form focused on word co-occurrence

$$\frac{(n_{ab} - N \cdot \frac{n_a}{N} \cdot \frac{n_b}{N})^2}{N \cdot \frac{n_a}{N} \cdot \frac{n_b}{N}} \stackrel{\text{rank}}{=} \frac{(n_{ab} - \frac{1}{N} \cdot n_a \cdot n_b)^2}{n_a \cdot n_b}$$

Association Measure Summary [2]

<i>Measure</i>	<i>Formula</i>
Mutual information (<i>MIM</i>)	$\frac{n_{ab}}{n_a \cdot n_b}$
Expected Mutual Information (<i>EMIM</i>)	$n_{ab} \cdot \log\left(N \cdot \frac{n_{ab}}{n_a \cdot n_b}\right)$
Chi-square (χ^2)	$\frac{(n_{ab} - \frac{1}{N} \cdot n_a \cdot n_b)^2}{n_a \cdot n_b}$
Dice's coefficient (<i>Dice</i>)	$\frac{n_{ab}}{n_a + n_b}$

Query Expansion Using Chain-of-Thought (CoT) Prompts ¹²

- Large Language Models (LLMs) can also be leveraged for query expansion by utilizing the knowledge embedded within them.
- Various prompts can be used to perform query expansion, e.g., zero-shot, few-shot and Chain-of-Thought (CoT).
- **CoT prompting** is a method to improve LLMs by prompting them to generate step-by-step reasoning before giving a final answer.
 - It is like showing your work in math by breaking complex problems into logical steps.
- **CoT Prompting Template**
 - Task: [e.g., Query expansion, or ranking comparison]
 - Input: [Query or ranking]
 - Step-by-step reasoning:
 - Step 1: Identify the main concepts or signals
 - Step 2: Analyze relationships or evidence
 - Step 3: Select or compute the final result
 - Final output: [Expanded query or ranking decision]

Example of CoT Prompting with Explicit Semantic Reasoning

Prompt:

Expand the following query for an IR system.
First, analyze the user's information need.
Then, reason step by step to identify synonyms, related concepts, and technical terms.
Finally, output a list of expansion terms.

Query:

NLP and Machine Learning

Output format:

Core concepts

Candidate expansion terms

Final selected expansion terms

” Ask ChatGPT

Core concepts

Natural Language Processing (NLP)

Machine Learning (ML)

Learning methods applied to language data

Candidate expansion terms

computational linguistics

text mining

language models

deep learning

neural networks

supervised learning

unsupervised learning

feature extraction

word embeddings

transformers

text classification

sequence modeling

Final selected expansion terms

natural language processing

machine learning

deep learning

neural networks

language models

word embeddings

transformers

3. Semi-Supervised Learning for Learning to Rank

14

- Formulating precise descriptions or narratives of users' information needs is inherently challenging. Even when linguists provide well-structured descriptions, the underlying conceptual models often remain incomplete.
 - For example, it may be unclear whether subtopics, such as technical espionage and industrial espionage, overlap or should be treated as distinct.
- In practice, however, users can usually identify a small set of documents that are relevant to their information needs.
- This observation motivates the learning-to-rank paradigm, which is typically framed as a supervised learning problem. In this setting, a user profile is learned from training data labeled either by human experts (e.g., relevance judgments or explicit relevance feedback) or generated automatically (e.g., pseudo-relevance feedback).
- The training data may consist of ranked lists with partial orderings or binary labels (e.g., “relevant” vs. “not relevant”) assigned to individual documents.
- Nevertheless, in real-world scenarios, the amount of relevance information available per query is often limited.

Semi-supervised learning

- Semi-supervised learning is a paradigm in machine learning that lies between supervised and unsupervised learning.
- It leverages a small amount of labeled data together with a large volume of unlabeled data to train a model. This approach is particularly valuable when labeled data are scarce or expensive to obtain, while unlabeled data are abundant.
- Filtering tasks can be naturally formulated as semi-supervised learning problems.
- Rather than requiring detailed descriptions or narratives of user information needs, they typically rely on a short initial query and, optionally, user-provided relevance feedback (or labels assigned by domain experts).
 - This yields a training set D , comprising a set of relevant documents D^+ and a set of irrelevant documents D^- , where D^+ is often small and D^- may even be empty.
 - The objective is to learn a representation, such as a set of discriminative features (e.g., term–weight pairs), that captures the underlying “topics of interest” or user information needs.

Example of User Relevance Feedback: A Training Set

<i>Doc</i>	<i>Keywords</i>	<i>Rel (label)</i>
d_1	GERMANY VW chief wants boost return sales tenfold	Yes
d_2	USA US Congress moves curb economic spying	Yes
d_3	US Bill ECONOM ECONOM ESPIONAG ECONOM	Yes
d_4	US Economic espionage bill clears Senate	Yes
d_5	GERMANY FOCUS Two men held VW industrial espionage	Yes
d_6	GERMAN GERMAN MAN VW ESPIONAG	Yes
d_7	GERMAN VW GERMAN VM	No
d_8	US ECONOM	No

Where, $D = \{d_1, d_2, \dots, d_8\}$, $D^+ = \{d_1, d_2, \dots, d_6\}$, and $D^- = \{d_7, d_8\}$

4. Rocchio-Based Information Filtering Model

- The **Rocchio algorithm**, a classical technique for query modification, is developed based on relevance feedback. It can be applied to **adaptive filtering** by initializing a query Q and iteratively updating it to an optimal query Q' using newly collected relevant documents.
- Optimal query
 - Maximizes the difference between the average vector representing the relevant documents and the average vector representing the non-relevant documents
- Modifies query according to

$$q'_j = \alpha \cdot q_j + \beta \cdot \frac{1}{|Rel|} \sum_{D_i \in Rel} d_{ij} - \gamma \cdot \frac{1}{|Nonrel|} \sum_{D_i \in Nonrel} d_{ij}$$

- α , β , and γ are parameters
 - Typical values 8, 16, 4
- $q_j \in Q$, is the initial weight of query term j ,
- d_{ij} is the weight (e.g., $\text{tf} \cdot \text{idf}$ weight) of the j th term in document i ,
- Rel is the set of identified relevant documents, e.g., $Rel = D^+ = \{d_1, d_2, \dots, d_6\}$
- $Nonrel$ is the set of non-relevant documents, e.g., $Nonrel = D^- = \{d_7, d_8\}$
- $|\cdot|$ gives the size of a set.

tf*idf Term Weighting

- It extends the **tf-idf weighting scheme** to characterize the importance of a term t_k with respect to an information need (or *Topics of Interest*), rather than an individual document.
 - Term frequency measures the importance of term $t_k \in T$ in relevant documents (D^+), where f_{ij} is term j 's frequency count in document i .

$$tf(t_k) = \frac{\sum_{d_i \in D^+} f_{ik}}{\sum_{d_i \in D^+} \sum_{t_j \in T} f_{ij}}$$

- Inverse document frequency measures importance in collection D :

$$idf(t_k) = \log \frac{N}{n_k}$$

where $N = |D|$, and $n_k = \{d_i \in D \mid t_k \in d_i\}$ is the number of documents that include t_k .

- Term weight function

$$w(t_k) = \frac{\beta}{|D^+|} tf(t_k) \times idf(t_k) - \frac{\gamma}{|D^-|} tf'(t_k) \times idf(t_k)$$

where tf' is the term frequency in D^- .

procedureTFIDFtermWeighting($D, D^+, D^-, \beta, \gamma$)

//Outputs: a set of terms T , and a function $w : T \rightarrow (-\infty, +\infty)$

(1) let $T = \emptyset, N = |D|, R = |D^+|, F = 0, F' = 0$;

for all $d_i \in D^+$ do

for all $t_k \in d_i$ do

$T = T \cup \{t_k\}$;

(2) for all $t_k \in T$ do

$\{tf(t_k) = 0, tf'(t_k) = 0, df(t_k) = 0\}$;

(3) for all $d_i \in D^+$ do

for all $t_k \in d_i$ do

$\{tf(t_k) = tf(t_k) + 1, F = F + 1\}$;

(4) for all $d_i \in D^-$ do

for all $t_k \in d_i$ do

$\{tf'(t_k) = tf'(t_k) + 1, F' = F' + 1\}$;

(5) for all $t_k \in T$ do

for all $d_i \in D$ do

if $\tau(t_k, d_i) = 1$ then $df(t_k) = df(t_k) + 1$;

(6) for all $t_k \in T$ do

$\{idf(t_k) = \log(\frac{N}{df(t_k)+0.5}), tf(t_k) = \frac{tf(t_k)}{F+0.5}, tf'(t_k) = \frac{tf'(t_k)}{F'+0.5},$

$w(t_k) = \frac{\beta}{R+0.5}tf(t_k) \times idf(t_k) - \frac{\gamma}{N-R+0.5}tf'(t_k) \times idf(t_k) \}$;

The Explanation of the Algorithm for calculating tf*idf term weights by using a training set:

- Step 1 & 2: Initialization. T is the set of selected features (or the initial long query), F is the sum of all terms' frequency counts in all relevant documents and F' is the sum of all terms' frequency counts in all non-relevant documents.
- Step 3 – 5: calculate tf, tf', F, F' and df.
- Step 6: uses the equation to calculate term weights.

PS: The Rocchio-based approach is adapted here to identify useful terms (i.e., topical features) based on tf*idf term weights. Note that only a training set is used in this process, and the initial query is ignored.

Rocchio-based TFIDF training algorithm

procedureTFIDFtraining($D, D^+, D^-, \beta, \gamma$)

- (1) call **procedure**TFIDFtermWeighting($D, D^+, D^-, \beta, \gamma$);
- (2) using function w to sort T in descending order;
- (3) select top- k terms and their weights.

- We assume that the above training algorithm can select the top- k terms to represent a user's information need. The parameter k can be determined via cross-validation.
- Given the selected terms, we next need to define how documents are ranked.
- Let T denote the set of selected terms and $w(\cdot)$ be the tf*idf weighting function. A ranking function for a document d can then be defined as follows.

$$\text{relevance}(d) = \sum_{t \in T} w(t) \tau(t, d), \quad \tau(t, d) = \begin{cases} 1 & \text{if } t \in d \\ 0 & \text{otherwise} \end{cases}$$

- Note that this ranking function differs from the abstract ranking model, as no explicit query is available in this setting.

CRICOS No.00213J

Testing Algorithm

procedure TFIDFtesting(T, w, U, δ)

// U is the set of incoming documents (or a testing set) and δ is a threshold.

(1) $Rel = \emptyset$;

(2) for all $d \in U$ do {

$rank(d) = 0$;

 for all $t_k \in T$ do

 if $\tau(t_k, d) = 1$ then $rank(d) = rank(d) + w(t_k)$;

 if $rank(d) \geq \delta$ then $Rel = Rel \cup \{d\}$; }

The algorithm applies a threshold δ to perform binary decision-making; however, it can be further adapted to generate a ranked output instead of a binary outcome.

5. Probabilistic Information Filtering Models

- **Probabilistic models** have gained significant attention in recent years because they form the basis for much of the work in Machine Learning, AI, and Data Analysis.
- They use *probable relevance* to describe the importance of term t_k for “Topics of Interest”.
- The individual term weight can be estimated based on two mutually exclusive independence assumptions:
 - **I1**: The distribution of terms in relevant documents is independent and their distribution in all documents is independent.
 - **I2**: The distribution of terms in relevant documents is independent and their distribution in non-relevant documents is independent.
- There are also two methods that refer to as ordering principles for presenting the result set:
 - **O1**: Probable relevance is based only on the presence of search terms in the documents.
 - **O2**: Probable relevance is based on both the presence of search terms in documents and their absence from documents.

Interpretations of Variables

<i>Doc</i>	<i>Keywords</i>	<i>Rel (label)</i>
d_1	GERMANY VW chief wants boost return sales tenfold	Yes
d_2	USA US Congress moves curb economic spying	Yes
d_3	US Bill ECONOM ECONOM ESPIONAG ECONOM	Yes
d_4	US Economic espionage bill clears Senate	Yes
d_5	GERMANY FOCUS Two men held VW industrial espionage	Yes
d_6	GERMAN GERMAN MAN VW ESPIONAG	Yes
d_7	GERMAN VW GERMAN VM	No
d_8	US ECONOM	No

- Let $N=|D|$ be the total number of documents in the training set D ;
 - $R = |D^+|$ be the number of relevant documents in D^+ for a topic;
 - $n(t)$ be the number of documents that contain term t ; and
 - $r(t)$ be the number of relevant documents that contain term t .
- For the training set D in the example, we have
 $N=8$, $R=6$, $n(\text{'US'})=4$, and $r(\text{'US'})=3$.

Weighting Formulas

- Choosing **I1** and **O1** yields a weighting function w_1 .
 - The ratio of the probability of occurrence in relevant documents to the probability of occurrence in the entire document set.
 - $w_1(t) = P(\text{Relevant} | t) = P(t | D^+) / P(t)$, where $P(t | D^+) = r(t)/R$, and $P(t) = n(t)/N$.
- Choosing **I2** and **O1** yields a weighting function w_2 .
 - The ratio of the probability of appearing in relevant documents to the probability of appearing in non-relevant documents

$$w_1(t) = \log \frac{\frac{r(t)}{R}}{\frac{n(t)}{N}}$$

$$w_2(t) = \log \frac{\frac{r(t)}{R}}{\frac{n(t) - r(t)}{N - R}}$$

Weighting Formulas cont.

- Choosing **I1** and **O2** yields a weighting function w_3 .
 - The ratio of the probability of being present in the relevant document to the probability of not being present in the relevant document.
 - The numerator $r(t) / (R - r(t)) = (r(t)/R) / ((R - r(t))/R)$
 - The denominator is the ratio of the probability of occurrence in all documents to the probability of occurrence in irrelevant documents = $(n(t)/N) / ((N - R)/N)$.
- Choosing **I2** and **O2** yields a weighting function w_4 .
 - The ratio of the probability of being present in the relevant document to the probability of not being present in the relevant document.
 - The denominator is the ratio of the probability of being present in irrelevant documents to the probability of not being present in all documents = $((n(t) - r(t))/N) / ((N - n(t)) - (R - r(t))) / N$,
 - where $n(t) - r(t)$ is the number of non-relevant documents that contain term t ; and
 - $(N - n(t)) - (R - r(t))$ is the number of non-relevant documents that don't contain term t .

$$w_3(t) = \log \frac{\frac{r(t)}{R - r(t)}}{\frac{n(t)}{N - R}}$$

$$w_4(t) = \log \frac{\frac{r(t)}{R - r(t)}}{\frac{n(t) - r(t)}{(N - n(t)) - (R - r(t))}}$$

6. BM25-based IF

- Most people believe that w_4 is most likely to yield the best results (BM25 like equation).
- The denominators in these fractions may be zero. Also considering the incomplete information in the training set, 0.5 is added to the weights to account for these problems.
- The modified weighting function appears as:

$$w_5(t) = \log \frac{\frac{r(t) + 0.5}{R - r(t) + 0.5}}{\frac{n(t) - r(t) + 0.5}{(N - n(t)) - (R - r(t)) + 0.5}}$$

BM25-based Training Algorithm

procedure *BM25termWeighting*(D, D^+, D^-)

(1) Let $T = \emptyset, N = |D|, R = |D^+|$

$T = \{t_k \in d_i, d_i \in D^+\}$ # a set comprehension to find all terms in D^+

(2) for all $t_k \in T$ do

$\{ n(t_k)=0, r(t_k)=0 \}$

(3) for all $t_k \in T$ do

for all $d_i \in D$ do

if $\tau(t_k, d_i)=1$ then $n(t_k) = n(t_k) + 1$

(4) for all $t_k \in T$ do

for all $d_i \in D^+$ do

if $\tau(t_k, d_i)=1$ then $r(t_k) = r(t_k) + 1$

(5) for all $t_k \in T$ do

$$w_5(t_k) = \left[\frac{r(t_k)+0.5}{R-r(t_k)+0.5} \right] \div \left[\frac{n(t_k)-r(t_k)+0.5}{(N-n(t_k))-(R-r(t_k))+0.5} \right]$$

The Explanation of BM25 Training Algorithm.

1. It uses a training set D to work out the BM25 like scores as probabilities; and use all terms in relevant documents.
2. Steps 2 to 4 calculate $r(t)$ and $n(t)$.
3. Step 5 calculates the final term weights

The algorithm extends BM25-based IR model for learning to rank if we can obtain or generate a training set.

7. LLM-Based Information Filtering

- Recent advances in NLP, particularly Large Language Models (LLMs) such as GPT-4, have shown strong potential for information filtering and personalized news recommendation [4].
- LLMs can be guided via prompts and text templates using various prompting strategies.
- A straightforward example is the input–output (IO) strategy, which elicits direct textual responses for information filtering or recommendation tasks.
- Retrieval-Augmented Generation (RAG) [5] has also gained popularity, combining LLMs with external knowledge bases, e.g., UMLS (Unified Medical Language System), to provide accurate, up-to-date information and transparency into the model’s generative process.

Example of Structured Filtering with Explanations: A Simple Prompt

You are an information filtering system. Determine whether the document satisfies the user's information need.

User need: "{UK: Britain's Channel 5 to broadcast Fashion Awards}"

Document: "{ Channel 5, Britain's newest terrestrial television channel, on Wednesday said it struck a three-year deal with the British Fashion Council for exclusive rights to broadcast the Lloyds Bank British Fashion Awards. "Fashion is now a defining part of British popular culture and this was an unmissable opportunity for 5 to give the British Fashion Awards its rightful place and profile on television," Adam Perry, Channel 5's controller of events, said in a statement. Initial, the independent production company that will produce coverage of the awards, said it aims to bring in top celebrities and world-class performers for the event. The British Fashion Awards will be held on October 22 at London's Royal Albert Hall. -- London Advertising Newsdesk +44 171 542 2815}"

Return the result strictly in valid JSON using the schema: { "relevant": boolean, "confidence": number, // between 0 and 1 "reason": string }

ChatGPT output:

```
{  
  "relevant": true,  
  "confidence": 0.92,  
  "reason": "The document explicitly reports  
that Britain's Channel 5 has secured exclusive  
rights to broadcast the British Fashion Awards,  
directly matching the information need about  
Channel 5 broadcasting the Fashion Awards."  
}
```

LLM-Based Information Filtering cont.

- LLMs have demonstrated strong capabilities in NLP. However, their effectiveness in news recommendation systems remains to be fully validated [4].
- Pros:
 - No need for labeled training data; informative prompts can guide the model.
 - LLMs can naturally generate explanations for recommendations, increasing transparency.
- Cons:
 - Prompting strategies can handle complex tasks but do not outperform well-trained deep neural models.
 - They require considerable human effort for prompt tuning and evaluation.
 - Evaluation of recommender systems is typically based on ranking metrics.
- Recent Development – RecPrompt [4]:
 - **Purpose:** A self-tuning prompting framework specifically designed for news recommendation.
 - **Mechanism:** Operates via iterative bootstrapping using two LLMs:
 - **News Recommender:** Generates recommendations using text prompts.
 - **Prompt Optimizer:** Refines prompts based on instructions to better align with user preferences and topics of interest.

References

- [1] Ted Kwartler, *Text Mining in Practice with R*, Wiley, 2017.
- [2] Chapter 6 in book - W. Bruce Croft, *Search Engines - Information retrieval in Practice*; Pearson, 2010.
- [3] Y. Li, A. Algarni, M. Albathan, Y. Shen and M. Bijaksana, Relevance Feature Discovery for Text Mining, *IEEE Transactions on Knowledge and Data Engineering*, 2015, vol. 27, no. 6, pages 1656-1669.
- [4] D. Liu, et al., RecPrompt: A Self-tuning Prompting Framework for News Recommendation Using Large Language Models, in CIKM '24: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, Pages 3902 – 3906, 2024, <https://doi.org/10.1145/3627673.3679987>
- [5] Kim Martineau, What is retrieval-augmented generation? <https://research.ibm.com/blog/retrieval-augmented-generation-RAG>